



Pashov Audit Group

Biconomy Security Review

May 14th 2026



Contents

1. About Pashov Audit Group	3
2. Disclaimer	3
3. Risk Classification	3
4. Methodology	4
5. About Composability and Nexus	4
6. Executive Summary	5
7. Centralization & Trust	7
8. Findings	8
Low findings	9
[L-01] Empty <code>OR</code> constraints always fail without an explicit invalid-OR error	9
[L-02] No clean way to ignore some returned fields	9
[L-03] Signed constraints treat large positive values as negative	10
[L-04] Nested-OR rejection is evaluation path dependent not structural	10
[L-05] <code>In</code> constraint silently misbehaves for signed reference data	11
[L-06] <code>Bytes32(c.referenceData)</code> accepts noncanonical encodings of constraint values	12
[L-07] <code>_validateConstraints()</code> may read beyond <code>rawValue</code> leading to zero-threshold issue	13



1. About Pashov Audit Group

Pashov Audit Group consists of 40+ freelance security researchers, who are well proven in the space - most have earned over \$100k in public contest rewards, are multi-time champions or have truly excelled in audits with us. We only work with proven and motivated talent.

With over 300 security audits completed — uncovering and helping patch thousands of vulnerabilities — the group strives to create the absolute very best audit journey possible. While 100% security is never possible to guarantee, we do guarantee you our team's best efforts for your project.

Check out our previous work [here](#) or reach out on Twitter [@pashovkrum](#).

2. Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs and on-chain monitoring are strongly recommended.

3. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users
- **Medium** - leads to a moderate material loss of assets in the protocol or moderately harms a group of users
- **Low** - leads to a minor material loss of assets in the protocol or harms a small group of users

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive



4. Methodology

Pashov Audit Group conducts each engagement as a team that works through the codebase end-to-end: we map the system, hunt for bugs, and then collaborate with the client to remediate the findings. This report tracks every commit reviewed, every finding raised, and the resolution status of each issue. Our auditors draw on experience from private engagements and public contests, and pick their own toolset per project — static analysis, fuzzers, formal verification, and AI-assisted review — based on what fits the codebase.

5. About Composability and Nexus

Biconomy is a modular account-abstraction infrastructure platform that lets users execute multi-step on-chain interactions via composable, type-safe transaction primitives. Its ERC-8211 composability library encodes argument constraints into auditable execution graphs and plugs into the Nexus smart account as a reusable submodule.

The report covers two related but independently usable systems. **ERC-8211** is a composable execution standard implemented as a standalone library that can be integrated into any contract. **Nexus/MEE/STX** is a tight integration of ERC-8211 into Biconomy's smart account and MEE execution stack.

ERC-8211 composability engine (standalone)

ComposableExecutionLib implements the ERC-8211 standard: batched operations declare parameter sources (`RAW_BYTES` , `STATIC_CALL` , or `BALANCE`) and attach constraints (`EQ` / `GTE` / `LTE` / `GTE_SIGNED` / `LTE_SIGNED` / `IN` / `SKIP` / `OR`) that are validated before each call. The library resolves inputs, executes the call, parses return data, and writes outputs to **ComposableStorage** for the next entry in the batch. It is designed for standalone use: `executeComposableDelegateCall()` can be called from any `msg.sender` , making ERC-8211 usable in isolation without the Nexus account stack.

Nexus / MEE / STX integration

When used within the Biconomy MEE stack, ERC-8211 is deployed as **ComposableExecutionModule** inside a **Nexus** ERC-7579 smart account. Users sign a single SuperTransaction hash that authorizes multi-step operations—deposit into Nexus, batch a composable sequence where each call's output feeds the next call's input, and withdraw—while the EntryPoint enforces gas accounting and MEE Nodes optionally sponsor execution costs. The module operates under hook pre/postCheck orchestration if a hook is installed via `withHook` . Users interact through **K1MeeValidator** in three signing modes: Simple (raw hash), On-Chain Tx (append hash to EIP-1559 tx), or ERC-2612 Permit (embed hash in deadline).



ERC-8211 components:

- **ComposableExecutionLib + ComposabilityDataTypes** — composability engine implementing ERC-8211; resolves `RAW_BYTES`, `STATIC_CALL`, `BALANCE` parameter sources; validates `EQ` / `GTE` / `LTE` / `GTE_SIGNED` / `LTE_SIGNED` / `IN` / `SKIP` / `OR` constraints; chains call outputs into inputs for the next batch step via **ComposableStorage**; callable from any `msg.sender` via `executeComposableDelegateCall()`

Nexus / MEE / STX components:

- **Nexus** — ERC-7579 modular account with UUPS upgradeability; installs/uninstalls validators, executors, hooks via EntryPoint-or-self gate; emergency hook uninstall with 1-day timelock escape hatch
- **K1MeeValidator** — tri-modal signature validator (Simple/OnChain/Permit); manages EOA owner and safe sender whitelist per account
- **ComposableExecutionModule** — Nexus executor module wrapping ComposableExecutionLib; entry point for ERC-4337 composable execution via `executeComposable()` with `withHook` orchestration
- **NexusAccountFactory** — deterministic CREATE2 deployment with batch module initialization via **NexusBootstrap** delegatecall
- **NodePaymaster** — gas sponsorship for MEE Nodes with `tx.origin`-based worker authorization

6. Executive Summary

A time-boxed security review of the `bcnmy/erc8211-contracts` and `bcnmy/stx-contracts` repositories was done by Pashov Audit Group, during which **Tejas Warambhe, Said, t.aksoy** engaged to review **Composability and Nexus**. A total of **7** issues were uncovered.

Protocol Summary

Project Name	Composability and Nexus
Protocol Type	Account Abstraction
Timeline	May 14th 2026

Review commit hashes:

- [c7ce1cb1970a5e40aa847b295459f114f3fb0db2](#)
(bcnmy/erc8211-contracts)
- [6c28f0b0c17b085bcb102bd4e7e7ac18ffd204e1](#)
(bcnmy/stx-contracts)



Fixes review commit hashes:

- [b731c5da12d3f3bad50bf798497a10c3316866f2](#)
(bcnmy/erc8211-contracts)
- [d0323ffea59894e1de4b46d3b1c9027acb1ab67c](#)
(bcnmy/stx-contracts)

Scope

ComposableExecutionLib.sol

ComposabilityDataTypes.sol

Nexus.sol



7. Centralization & Trust

The protocol has 3 privileged roles:

- **Nexus Account Owner** — installs/uninstalls validators, executors, hooks, and fallbacks via `installModule` / `uninstallModule` (onlyEntryPointOrSelf gate); emergency hook uninstall with 1-day timelock escape hatch via `emergencyUninstallHook`; UUPS upgrade authorization via `upgradeToAndCall`
- **MEE Node Operator** — submits UserOps to EntryPoint on behalf of users; operates NodePaymaster restricted to master EOA plus whitelisted worker EOAs via `tx.origin` check; whitelists workers via `whitelistWorkerEOA` (owner-only, no delay); manages EntryPoint deposits via `deposit` / `withdrawTo`
- **Factory Owner** — controls EntryPoint staking for factory via `addStake` , `unlockStake` , `withdrawStake` (Stakeable); no authority over deployed accounts post-creation



8. Findings

Findings count

Severity	Amount
Low	7
Total findings	7

Summary of findings

ID	Title	Severity	Status
[L-01]	Empty <code>OR</code> constraints always fail without an explicit invalid-OR error	Low	Resolved
[L-02]	No clean way to ignore some returned fields	Low	Resolved
[L-03]	Signed constraints treat large positive values as negative	Low	Resolved
[L-04]	Nested-OR rejection is evaluation path dependent not structural	Low	Resolved
[L-05]	In constraint silently misbehaves for signed reference data	Low	Resolved
[L-06]	Bytes32(c.referenceData) accepts noncanonical encodings of constraint values	Low	Resolved
[L-07]	<code>_validateConstraints()</code> may read beyond <code>rawValue</code> leading to zero-threshold issue	Low	Resolved



Low findings

[L-01] Empty **OR** constraints always fail without an explicit invalid-OR error

Description

The PR adds `ConstraintType.OR` where `referenceData` is `abi.encode(Constraint[])`. The implementation accepts an encoded empty array, but such a constraint can never pass:

```
Constraint[] memory subs = abi.decode(c.referenceData, (Constraint[]));
uint256 subsLen = subs.length;
bool anyMet;
for (uint256 j; j < subsLen; j++) {
    if (_checkConstraint(value, subs[j])) {
        anyMet = true;
        break;
    }
    unchecked {
        ++j;
    }
}
if (!anyMet) revert ConstraintNotMet(c.constraintType);
```

If `subs.length == 0`, the loop is skipped and `anyMet` remains false, so the function reverts with `ConstraintNotMet(OR)`.

Nested OR rejection is intentionally covered by PR tests, but empty OR is not explicitly covered. An explicit error would make malformed OR payloads easier for wallets and users to diagnose.

Consider rejecting empty OR constraints explicitly before evaluating sub-constraints.

[L-02] No clean way to ignore some returned fields

Description

Constraints are checked by position in the returned data. The first constraint checks the first 32-byte field, the second checks the second field, and so on. If a user wants to check only the 3rd returned field but ignore the 1st and 2nd, they still need to add dummy always-true constraints in those earlier positions to keep everything aligned.

A user may route through multiple swaps before a final deposit and only care that the final amount is good enough, not every intermediate field or quote component returned along the way. The current model works well when skipping validation for whole steps, but it is clunky when skipping only some fields inside one multi-word result.

Add a SKIP constraint type that unconditionally returns true, making intent explicit:



[L-03] Signed constraints treat large positive values as negative

Description

GTE_SIGNED / LTE_SIGNED does: `int256(uint256(value))`

GTE_SIGNED / LTE_SIGNED compare values using: `int256(uint256(value))`

This is correct when the resolved value is genuinely meant to be interpreted as an int256, but it also means any value $\geq 2^{255}$ is treated as negative under two's complement rules. So a very large positive uint256 is no longer "large positive" once it enters the signed path.

This applies not only to direct RAW_BYTES inputs, but also to values coming from: `(bool success, bytes memory returnData) = contractAddr.staticcall(callData);` If a STATIC_CALL returns a 32-byte value above int256.max, the signed constraint logic will also interpret it as negative.

Because of that, signed constraints should only be used when the resolved input or returned value is known to be in the signed int256 domain. In practice, callers should avoid using GTE_SIGNED / LTE_SIGNED for values that may exceed $2^{255} - 1$, and this expectation should be documented clearly. If stricter handling is desired, the implementation could additionally reject signed comparisons when the resolved value is outside the intended signed domain.

[L-04] Nested-OR rejection is evaluation path dependent not structural

Description

The NatSpec at `ComposableExecutionLib::_validateConstraints()` promises that nested OR is rejected to keep what the user signs flat. The `InvalidConstraintType()` revert fires only when execution actually reaches a nested-OR sub. The outer loop short-circuits on the first matching sub:

```
function _validateConstraints(bytes memory rawValue, Constraint[] calldata constraints)
private pure {
    uint256 len = constraints.length;
    for (uint256 i; i < len;) {
        Constraint memory c = constraints[i];
        bytes32 value;
        assembly {
            value := mload(add(rawValue, add(0x20, mul(i, 0x20))))
        }
        if (c.constraintType == ConstraintType.OR) {
            Constraint[] memory subs = abi.decode(c.referenceData, (Constraint[]));
            uint256 subsLen = subs.length;
            bool anyMet;
            for (uint256 j; j < subsLen;) {
                if (_checkConstraint(value, subs[j])) {
                    anyMet = true;
                    break;
                }
            }
            // <-- later subs (incl. a nested OR) never
```



```
evaluated
    }
    unchecked {
        ++j;
    }
}
if (!anyMet) revert ConstraintNotMet(c.constraintType);
} else {
    if (!_checkConstraint(value, c)) revert ConstraintNotMet(c.constraintType);
}
unchecked {
    ++i;
}
}
}
```

The Off-chain tooling that walks the constraint tree to render "what you are signing" sees a nested `OR`, and the transaction may or may not be rejected due to it, depending on runtime data. This breaks the purpose of displaying accurate signing data and will be a burden for to handle off-chain.

It is recommended to not allow a nested `OR` in any case by pre-passing over all subs.

[L-05] In constraint silently misbehaves for signed reference data

Description

The `ConstraintType.IN` is documented as "suitable for unsigned ranges and same-sign signed ranges", but nothing in the code enforces that contract. The handler decodes both bounds as `bytes32` and compares with the EVM's bitwise (unsigned) `>=` / `<=`:

```
// Constraint type for parameter validation
enum ConstraintType {
    EQ, // Equal to (bitwise equality; suitable for signed, unsigned, addresses, bytes32)
    GTE, // Greater than or equal to (unsigned)
    LTE, // Less than or equal to (unsigned)
    IN, // In range [lower, upper] (bytes32 comparison; suitable for unsigned ranges and
    same-sign signed ranges) <<@
    GTE_SIGNED, // Greater than or equal to (signed int256)
    LTE_SIGNED, // Less than or equal to (signed int256)
    OR // At least one sub-constraint must pass; referenceData = abi.encode(Constraint[]);
    sub-constraints must be leaf types (no nested OR)
}
```

```
function _checkConstraint(bytes32 value, Constraint memory c) private pure returns
(bool) {
    ConstraintType ct = c.constraintType;
    // . . .
} else if (ct == ConstraintType.IN) {
    (bytes32 lower, bytes32 upper) = abi.decode(c.referenceData, (bytes32,
bytes32));
    <<@
    return value >= lower && value <= upper;
```



```

    } else if (ct == ConstraintType.GTE_SIGNED) {
        // . . .
    }

```

Because a negative `int256` has its high bit set, its `bytes32` form is a huge `uint256`. Three silent failure modes follow:

1. Range straddling zero, e.g. `IN(-10, 10)`: the negative lower bound decodes to a huge unsigned value, so `lower > upper` and the predicate is unsatisfiable. The signer expects "small values around zero," but every batch reverts. Same for any mixed-sign range, including `IN(-N, 0)`.
2. The signer accidentally swaps the bounds, e.g. `IN(10, -10)`. They expect this to be either rejected or behave like the previous case. Instead, the lower bound is the small number 10 and the upper bound is the huge unsigned value that -10 decodes to, so the accepted set widens to every value whose magnitude is at least 10, both large positives and large-magnitude negatives. The constraint is now fail-open and authorizes far more than the signer thought they were signing.
3. The signer writes a same-sign signed range but in descending order, e.g. `IN(-10, -100)`. The comment says same-sign signed ranges are fine, so they assume order does not matter. But because the comparison is unsigned, only the ascending form `IN(-100, -10)` actually works; the descending form gives `lower > upper` and, like case 1, rejects every value. This ordering requirement is nowhere in the documentation.

Cases 1 and 3 brick the input parameter (every batch reverts with `ConstraintNotMet(IN)`). However, case 2 is concerning as a signer who reasonably assumes `IN` is signed-aware and types `IN(10, -10)` silently authorizes everything outside `(-10, 10)`.

It is recommended to add `if (lower > upper) revert InvalidConstraintRange();` inside the `IN` branch. This resolves case 2 and converts cases 1/3 into an immediate, attributable revert instead of a silent empty set. Alternatively, add a dedicated `IN_SIGNED` constraint type that performs signed `int256` range comparison, so that callers working with signed values are not required to use the unsigned `IN` operator.

[L-06] Bytes32(c.referenceData) accepts noncanonical encodings of constraint values

Description

The `ComposableExecutionLib::_checkConstraint()` uses `bytes32(c.referenceData)` casting to read the first 32 bytes of `referenceData`:

```

function _checkConstraint(bytes32 value, Constraint memory c) private pure returns
(bool) {
    ConstraintType ct = c.constraintType;
    if (ct == ConstraintType.EQ) {
        return value == bytes32(c.referenceData);    <<@
    } else if (ct == ConstraintType.GTE) {
        return value >= bytes32(c.referenceData);    <<@
    } else if (ct == ConstraintType.LTE) {
        return value <= bytes32(c.referenceData);    <<@
    }
}

```



```

    } else if (ct == ConstraintType.IN) {
        (bytes32 lower, bytes32 upper) = abi.decode(c.referenceData, (bytes32, bytes32));
        return value >= lower && value <= upper;
    } else if (ct == ConstraintType.GTE_SIGNED) {
        return int256(uint256(value)) >=
int256(uint256(bytes32(c.referenceData))); <<@
    } else if (ct == ConstraintType.LTE_SIGNED) {
        return int256(uint256(value)) <=
int256(uint256(bytes32(c.referenceData))); <<@
    } else {
        revert InvalidConstraintType();
    }
}

```

If `referenceData.length != 32`, the cast left-aligns whatever is present and (deterministically) zero-pads the remaining bytes from the ABI's 32-byte calldata alignment.

The `IN` branch is already safe because it goes through `abi.decode(c.referenceData, (bytes32, bytes32))`, which reverts strictly on length mismatch; however, the five other branches do not.

A signer/integrator who builds a constraint with a non-canonical encoding gets a silently wrong comparison instead of a revert.

It is recommended to add an explicit `require(c.referenceData.length == 32)` in the five leaf branches of `_checkConstraint()` so that encoding mistakes revert with a clear error instead of silently miscomparing.

To illustrate the alignment issue: `bytes32(abi.encode(uint256(5)))` produces `0x0000000000000000000000000000000000000000000000000000000000000005` (correct, right-aligned), whereas `bytes32(abi.encodePacked(uint8(5)))` produces `0x0500000000000000000000000000000000000000000000000000000000000000` (left-aligned, wrong). Similarly, `bytes32(abi.encode(int256(-1)))` gives the expected `0xffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff`, but `bytes32(abi.encodePacked(uint8(0xff)))` gives `0xff00000000000000000000000000000000000000000000000000000000000000`, which is interpreted as an extremely large negative int256 under the signed constraints.

[L-07] `_validateConstraints()` may read beyond `rawValue` leading to zero-threshold issue

Description

The loop inside `ComposableExecutionLib::_validateConstraints()` reads `constraints.length` words from `rawValue` with no check that `rawValue.length >= constraints.length * 32`:

```

function _validateConstraints(bytes memory rawValue, Constraint[] calldata constraints)
private pure {
    uint256 len = constraints.length;

```



```
for (uint256 i; i < len;) {
    Constraint memory c = constraints[i];
    bytes32 value;
    assembly {
        value := mload(add(rawValue, add(0x20, mul(i, 0x20)))) <<@
    }
    // . . .
}
```

When `rawValue` is shorter than `constraints.length * 32`, the read lands in adjacent memory rather than reverting. The bug surfaces at all three callers of `_validateConstraints()`:

- `STATIC_CALL`: a static call to an EOA, a not-yet-deployed CREATE2 address, or a contract upgraded to return less data succeeds with `returndatasize = 0`.
- `BALANCE`: `abi.encode(balance)` is exactly one 32-byte word, but `constraints.length > 1` is not rejected.
- `RAW_BYTES`: Signer supplied.

The out-of-bounds read is not zero. The loop allocates `Constraint memory c` before the assembly block, and that struct's first word, its `constraintType` enum byte, which lands at exactly the address the `mload` reads. So the phantom value is the constraint's own type index (`EQ = 0`, `GTE = 1`, `GTE_SIGNED = 4`, ...).

That silently satisfies natural zero-threshold predicates: `EQ(0)`, `GTE(0)`, and `GTE_SIGNED(0)` all pass against empty returndata. A signed execution whose only guard is a silently passing predicate (e.g. an oracle sanity check `GTE_SIGNED(0)`) is approved as if the staticcall had returned a satisfying value, bypassing the signer's intended precondition.

Recommendations

It is recommended to check for the `rawValue` length to be greater than or equal to `constraints.length * 32`. Also, reject `constraints.length > 1` on the `BALANCE` path, since balance is a single word by construction.